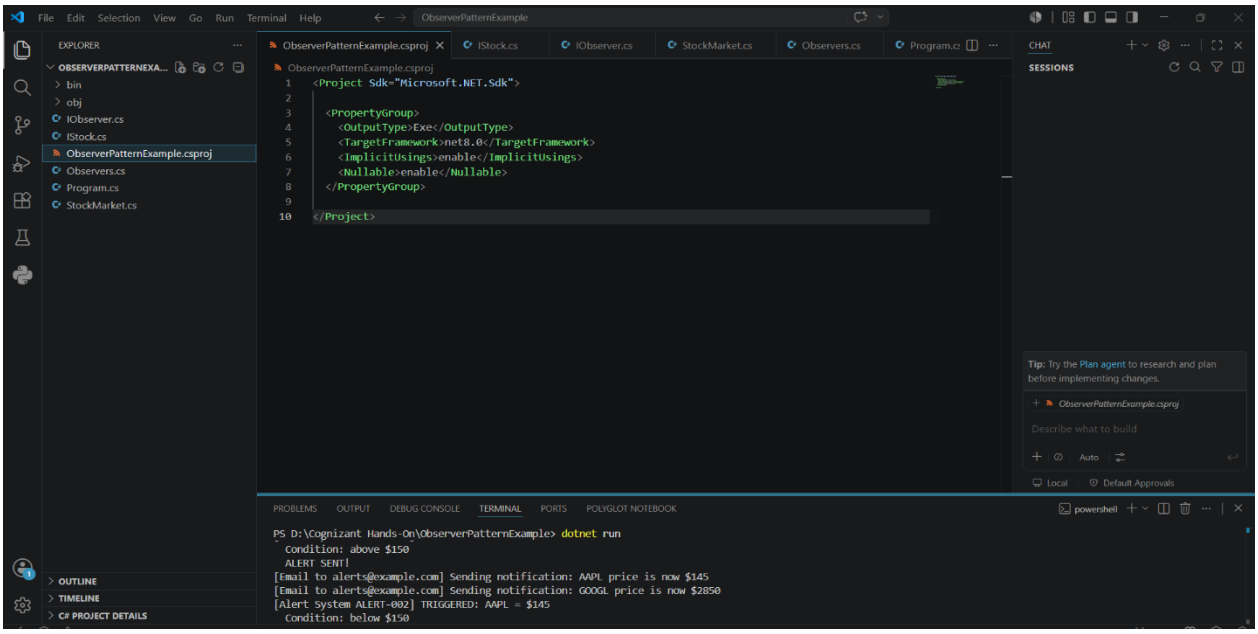
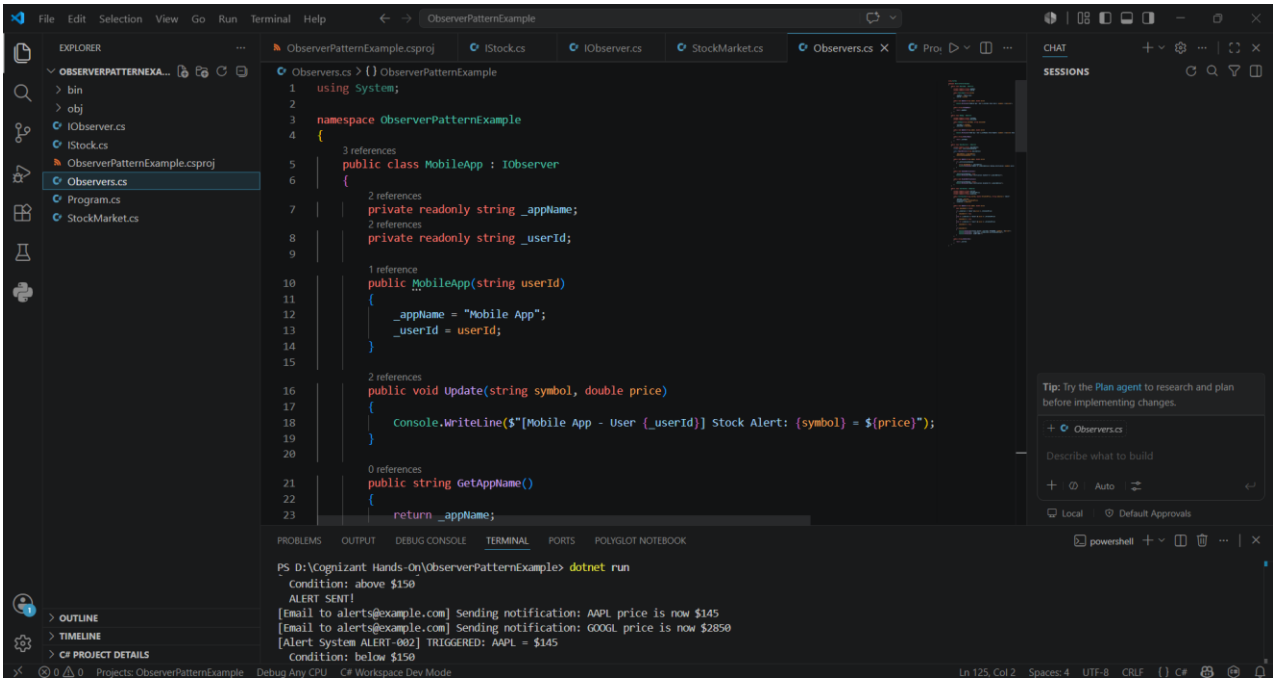
Hands-On

Exercise 7: Implementing the Observer Pattern

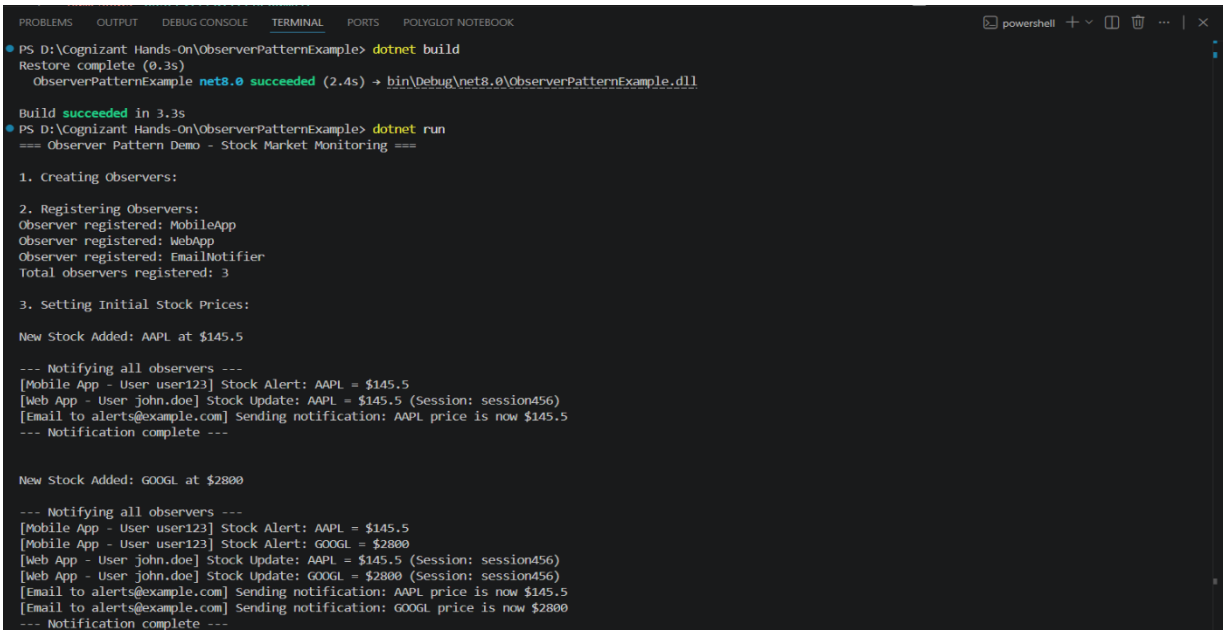
In this module, I successfully implemented, executed, and tested the Observer Pattern in .NET by developing a project named ObserverPatternExample. I created a Stock interface with methods to register, deregister, and notify observers, and implemented a StockMarket class that maintained a list of observers and managed stock price updates. I also defined an Observer interface with an Update() method and developed concrete observer classes such as MobileApp and WebApp to receive notifications whenever stock prices changed. After completing the implementation, I executed the program and tested the registration, deregistration, and notification mechanisms to verify that all subscribed observers received updates automatically whenever stock data was modified. This hands-on exercise helped me understand how the Observer Pattern enables efficient one-to-many communication between objects, promotes loose coupling, and ensures that dependent components remain synchronized with changes in .NET applications.

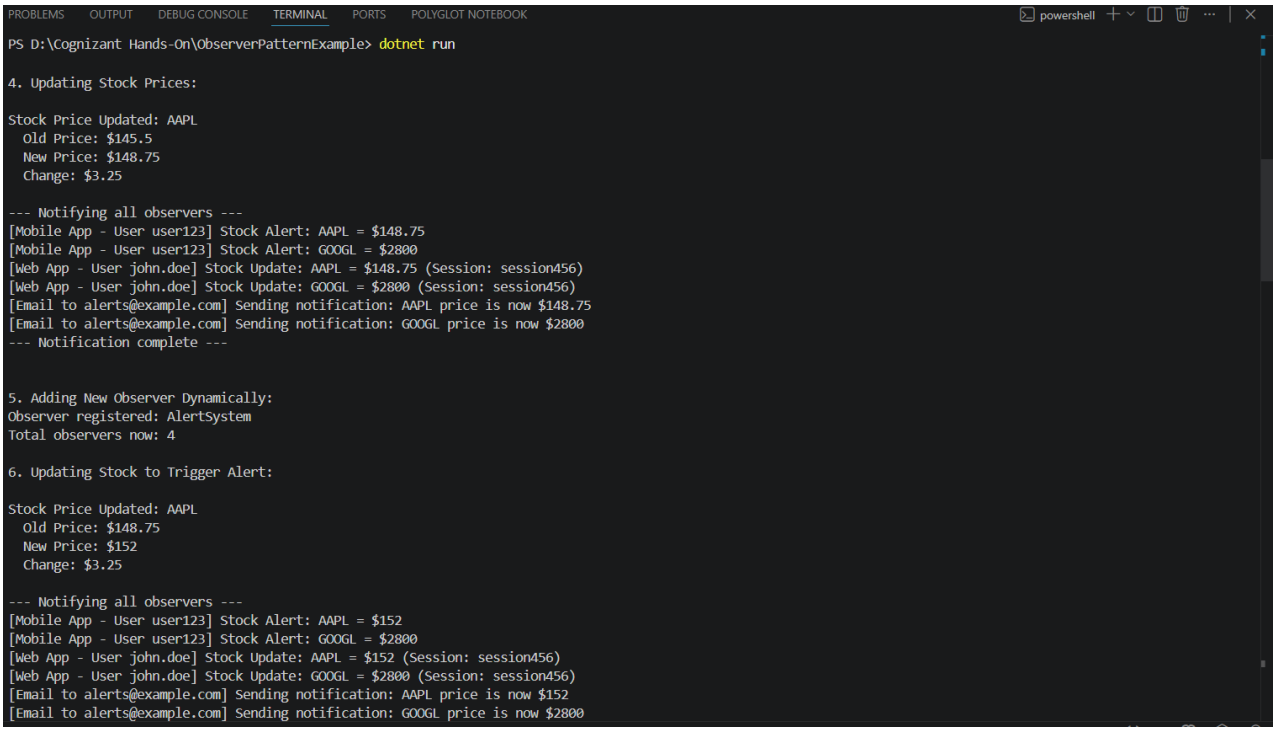
Coding:

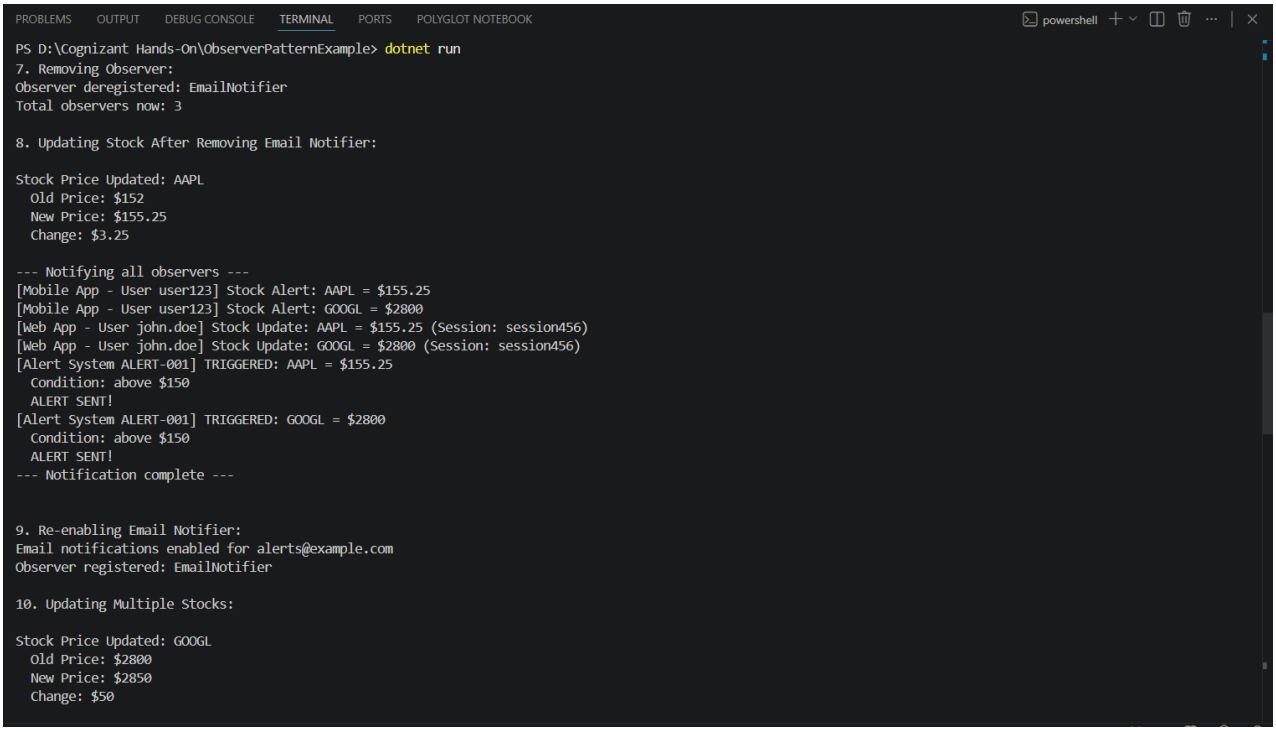


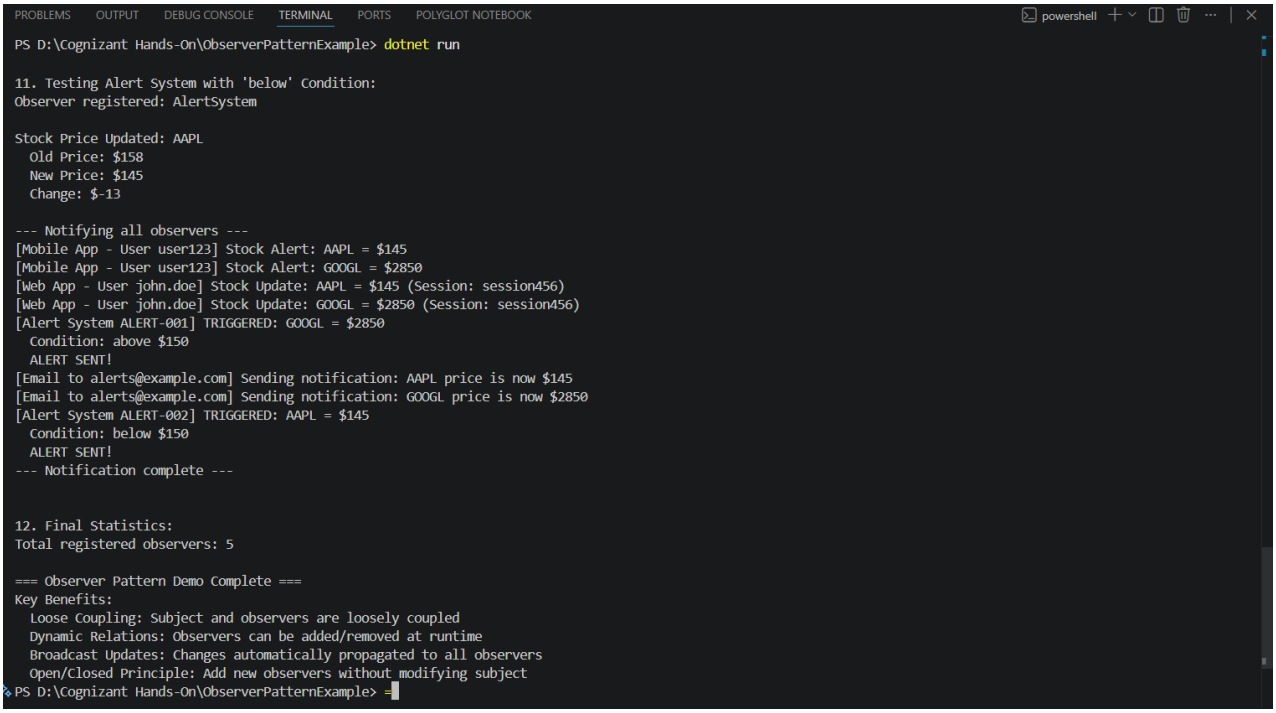

File Edit Selection View Go Run Terminal Help
ObserverPatternExample
EXPLORER
OBSERVERPATTERNEXA...
bin
obj
IObserver.cs
IStock.cs
ObserverPatternExample.csproj
Observers.cs
Program.cs
StockMarket.cs
ObserverPatternExample.csproj
IStock.cs
IObserver.cs
StockMarket.cs
Observers.cs
Observers.cs > { } ObserverPatternExample
using System;
namespace ObserverPatternExample
{
3 references
public class MobileApp : IObserver
{
2 references
private readonly string _appName;
2 references
private readonly string _userId;
1 reference
public MobileApp(string userId)
{
_appName = "Mobile App";
_userId = userId;
}
2 references
public void Update(string symbol, double price)
{
Console.WriteLine($"[Mobile App - User {_userId}] Stock Alert: {symbol} = ${price}");
}
0 references
public string GetAppName()
{
return _appName;
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
Observers.cs
Describe what to build
Auto
Local
Default Approvals
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POLYGLOT NOTEBOOK
powershell
PS D:\Cognizant Hands-On\ObserverPatternExample> dotnet run
Condition: above $150
ALERT SENT!
[Email to alerts@example.com] Sending notification: AAPL price is now $145
[Email to alerts@example.com] Sending notification: GOOGL price is now $2850
[Alert System ALERT-002] TRIGGERED: AAPL = $145
Condition: below $150
OUTLINE
TIMELINE
C# PROJECT DETAILS

Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POLYGLOT NOTEBOOK
powershell
PS D:\Cognizant Hands-On\ObserverPatternExample> dotnet build
Restore complete (0.3s)
ObserverPatternExample net8.0 succeeded (2.4s) → bin\Debug\net8.0\ObserverPatternExample.dll
Build succeeded in 3.3s
PS D:\Cognizant Hands-On\ObserverPatternExample> dotnet run
=== Observer Pattern Demo - Stock Market Monitoring ===
1. Creating Observers:
2. Registering Observers:
Observer registered: MobileApp
Observer registered: WebApp
Observer registered: EmailNotifier
Total observers registered: 3
3. Setting Initial Stock Prices:
New Stock Added: AAPL at $145.5
--- Notifying all observers ---
[Mobile App - User user123] Stock Alert: AAPL = $145.5
[Web App - User john.doe] Stock Update: AAPL = $145.5 (Session: session456)
[Email to alerts@example.com] Sending notification: AAPL price is now $145.5
--- Notification complete ---
New Stock Added: GOOGL at $2800
--- Notifying all observers ---
[Mobile App - User user123] Stock Alert: AAPL = $145.5
[Mobile App - User user123] Stock Alert: GOOGL = $2800
[Web App - User john.doe] Stock Update: AAPL = $145.5 (Session: session456)
[Web App - User john.doe] Stock Update: GOOGL = $2800 (Session: session456)
[Email to alerts@example.com] Sending notification: AAPL price is now $145.5
[Email to alerts@example.com] Sending notification: GOOGL price is now $2800
--- Notification complete ---


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POLYGLOT NOTEBOOK
powershell
PS D:\Cognizant Hands-On\ObserverPatternExample> dotnet run
4. Updating Stock Prices:
Stock Price Updated: AAPL
Old Price: $145.5
New Price: $148.75
Change: $3.25
--- Notifying all observers ---
[Mobile App - User user123] Stock Alert: AAPL = $148.75
[Mobile App - User user123] Stock Alert: GOOGL = $2800
[Web App - User john.doe] Stock Update: AAPL = $148.75 (Session: session456)
[Web App - User john.doe] Stock Update: GOOGL = $2800 (Session: session456)
[Email to alerts@example.com] Sending notification: AAPL price is now $148.75
[Email to alerts@example.com] Sending notification: GOOGL price is now $2800
--- Notification complete ---
5. Adding New Observer Dynamically:
Observer registered: AlertSystem
Total observers now: 4
6. Updating Stock to Trigger Alert:
Stock Price Updated: AAPL
Old Price: $148.75
New Price: $152
Change: $3.25
--- Notifying all observers ---
[Mobile App - User user123] Stock Alert: AAPL = $152
[Mobile App - User user123] Stock Alert: GOOGL = $2800
[Web App - User john.doe] Stock Update: AAPL = $152 (Session: session456)
[Web App - User john.doe] Stock Update: GOOGL = $2800 (Session: session456)
[Email to alerts@example.com] Sending notification: AAPL price is now $152
[Email to alerts@example.com] Sending notification: GOOGL price is now $2800


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POLYGLOT NOTEBOOK
powershell
PS D:\Cognizant Hands-On\ObserverPatternExample> dotnet run
7. Removing Observer:
Observer deregistered: EmailNotifier
Total observers now: 3
8. Updating Stock After Removing Email Notifier:
Stock Price Updated: AAPL
Old Price: $152
New Price: $155.25
Change: $3.25
--- Notifying all observers ---
[Mobile App - User user123] Stock Alert: AAPL = $155.25
[Mobile App - User user123] Stock Alert: GOOGL = $2800
[Web App - User john.doe] Stock Update: AAPL = $155.25 (Session: session456)
[Web App - User john.doe] Stock Update: GOOGL = $2800 (Session: session456)
[Alert System ALERT-001] TRIGGERED: AAPL = $155.25
Condition: above $150
ALERT SENT!
[Alert System ALERT-001] TRIGGERED: GOOGL = $2800
Condition: above $150
ALERT SENT!
--- Notification complete ---
9. Re-enabling Email Notifier:
Email notifications enabled for alerts@example.com
Observer registered: EmailNotifier
10. Updating Multiple Stocks:
Stock Price Updated: GOOGL
Old Price: $2800
New Price: $2850
Change: $50


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POLYGLOT NOTEBOOK
powershell
PS D:\Cognizant Hands-On\ObserverPatternExample> dotnet run
11. Testing Alert System with 'below' Condition:
Observer registered: AlertSystem
Stock Price Updated: AAPL
Old Price: $158
New Price: $145
Change: $-13
--- Notifying all observers ---
[Mobile App - User user123] Stock Alert: AAPL = $145
[Mobile App - User user123] Stock Alert: GOOGL = $2850
[Web App - User john.doe] Stock Update: AAPL = $145 (Session: session456)
[Web App - User john.doe] Stock Update: GOOGL = $2850 (Session: session456)
[Alert System ALERT-001] TRIGGERED: GOOGL = $2850
Condition: above $150
ALERT SENT!
[Email to alerts@example.com] Sending notification: AAPL price is now $145
[Email to alerts@example.com] Sending notification: GOOGL price is now $2850
[Alert System ALERT-002] TRIGGERED: AAPL = $145
Condition: below $150
ALERT SENT!
--- Notification complete ---
12. Final Statistics:
Total registered observers: 5
=== Observer Pattern Demo Complete ===
Key Benefits:
Loose Coupling: Subject and observers are loosely coupled
Dynamic Relations: Observers can be added/removed at runtime
Broadcast Updates: Changes automatically propagated to all observers
Open/Closed Principle: Add new observers without modifying subject
PS D:\Cognizant Hands-On\ObserverPatternExample> =